

# ITA0613

## Machine Learning

Name : Chiranjibi Pradhan

Reg No : 192511475

### 1. FIND-S Algorithm

#### Aim

To implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis from a given set of training examples.

#### Algorithm

1. Initialize the most specific hypothesis.
2. Read the training examples.
3. Consider only positive examples.
4. Compare attributes with the current hypothesis.
5. If attributes differ, replace with '?'.
6. Continue until all examples are processed.
7. Display the final hypothesis.

#### Python Code

```
import pandas as pd
```

```
data = pd.read_csv("trainingdata.csv")
```

```
concepts = data.iloc[:, :-1].values
```

```
target = data.iloc[:, -1].values
```

```
hypothesis = concepts[0].copy()
```

```

for i in range(len(concepts)):
    if target[i] == "Yes":
        for j in range(len(hypothesis)):
            if concepts[i][j] != hypothesis[j]:
                hypothesis[j] = '?'

print("Final Hypothesis:")
print(hypothesis)

```

## Output

```

... Initial Hypothesis:
    ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

    Final Hypothesis:
    ['Sunny' 'Warm' '?' 'Strong' '?' '?']

```

+ Code
+ Text

## Result

The most specific hypothesis obtained using the FIND-S algorithm is:

<Sunny, Warm, ?, Strong, ?, ?>

## Conclusion

The FIND-S algorithm successfully identified the most specific hypothesis that covers all positive training examples while ignoring negative examples.

## 2. Candidate Elimination Algorithm

## **Aim**

To implement the Candidate Elimination Algorithm and determine the version space of hypotheses consistent with the training examples.

## **Algorithm**

1. Initialize Specific Boundary (S).
2. Initialize General Boundary (G).
3. For positive examples:
  - Generalize S.
  - Remove inconsistent hypotheses from G.
4. For negative examples:
  - Specialize G.
  - Remove inconsistent hypotheses from S.
5. Continue for all examples.
6. Display S and G.

## **Python Code**

```
import pandas as pd
```

```
data = pd.read_csv("trainingdata.csv")
```

```
concepts = data.iloc[:, :-1].values
```

```
target = data.iloc[:, -1].values
```

```
specific = concepts[0].copy()
```

```
general = [["?" for i in range(len(specific))]  
           for j in range(len(specific))]
```

```
for i, h in enumerate(concepts):

    if target[i] == "Yes":

        for x in range(len(specific)):

            if h[x] != specific[x]:

                specific[x] = '?'

                general[x][x] = '?'

    else:

        for x in range(len(specific)):

            if h[x] != specific[x]:

                general[x][x] = specific[x]

            else:

                general[x][x] = '?'

print("Specific Boundary:")
print(specific)

print("General Boundary:")
for g in general:
    print(g)
```

**Output**

```

... Initial Specific Hypothesis:
    ['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']

Initial General Hypothesis:
[['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?'], ['?', '?', '?', '?', '?', '?']]

Final Specific Hypothesis:
['Sunny' 'Warm' '?' 'Strong' '?' '?']

Final General Hypothesis:
['Sunny', '?', '?', '?', '?', '?']
['?', 'Warm', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']
['?', '?', '?', '?', '?', '?']

```

## Result

The Specific and General boundaries representing the version space are obtained.

## Conclusion

Candidate Elimination effectively determines all hypotheses consistent with the training data.

## 3. ID3 Decision Tree Algorithm

### Aim

To implement the ID3 algorithm and classify a new sample using a decision tree.

### Algorithm

1. Calculate entropy.
2. Compute information gain.
3. Select the attribute with maximum information gain.
4. Create root node.
5. Repeat recursively for child nodes.

6. Stop when all samples belong to one class.
7. Classify new samples.

## Python Code

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv("playtennis.csv")

X = data.iloc[:, :-1]
y = data.iloc[:, -1]

X = pd.get_dummies(X)

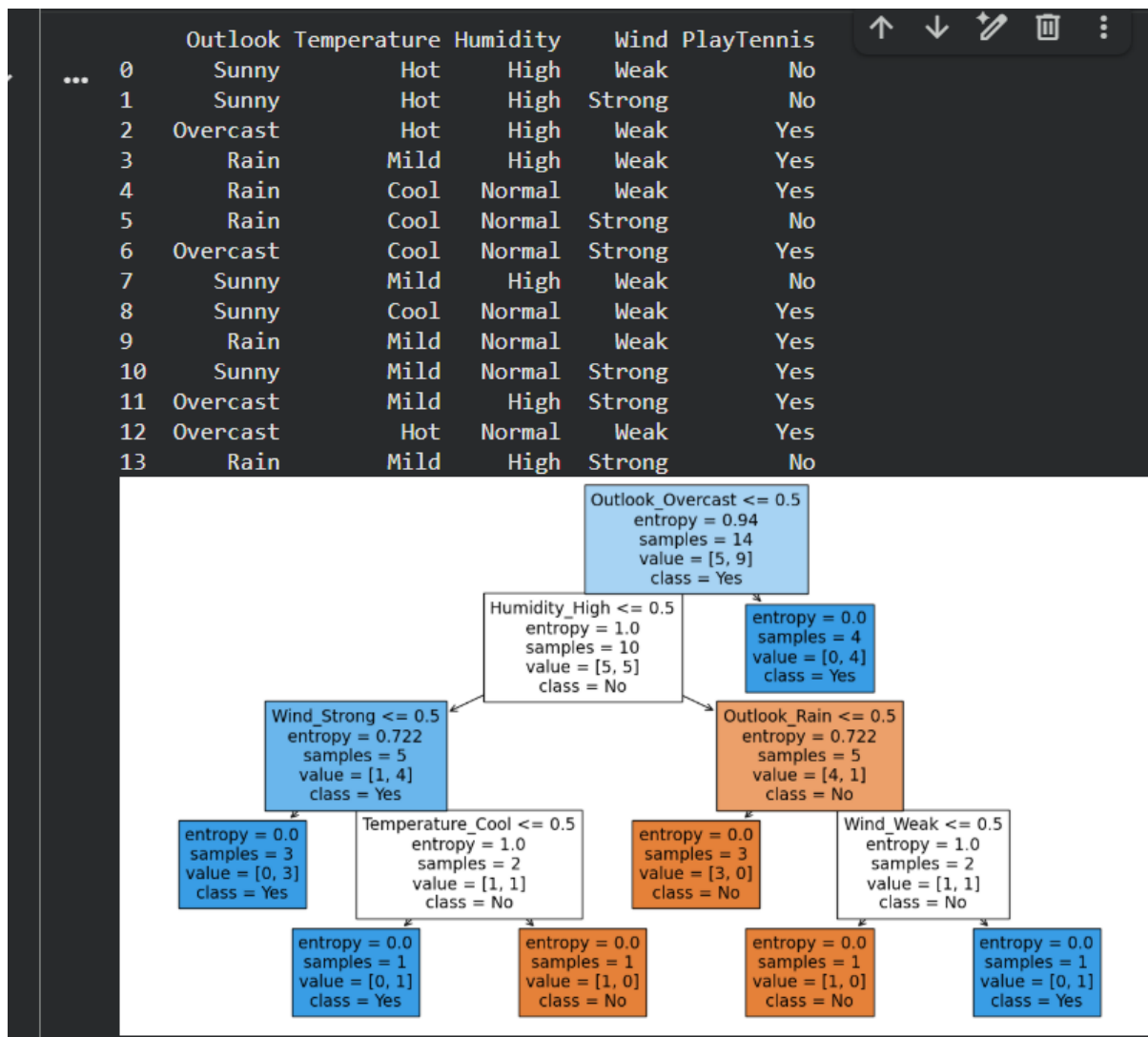
clf = DecisionTreeClassifier(criterion='entropy')
clf.fit(X, y)

plt.figure(figsize=(10,6))
tree.plot_tree(clf, filled=True)
plt.show()

sample = [1,0,0,1,0,1,0,1,0,0]
prediction = clf.predict([sample])

print("Prediction:", prediction)
```

## Output



## Result

The ID3 decision tree is generated and used to classify new samples.

## Conclusion

The ID3 algorithm accurately classifies data using information gain and entropy.

## 4. Artificial Neural Network using Backpropagation

### Aim

To build an Artificial Neural Network using the Backpropagation algorithm and evaluate its performance.

### Algorithm

1. Initialize weights randomly.
2. Perform forward propagation.
3. Calculate error.
4. Perform backward propagation.
5. Update weights.
6. Repeat until convergence.
7. Test the network.

### **Python Code**

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

mlp = MLPClassifier(hidden_layer_sizes=(5,),
                    max_iter=1000)

mlp.fit(X_train, y_train)

accuracy = mlp.score(X_test, y_test)
```



```
print("Accuracy:", accuracy)
```

## Output

```
*** Artificial Neural Network using Backpropagation
=====
Training Samples : 105
Testing Samples  : 45

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy = 100.0 %

Classification Report

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 19      |
| 1            | 1.00      | 1.00   | 1.00     | 13      |
| 2            | 1.00      | 1.00   | 1.00     | 13      |
| accuracy     |           |        | 1.00     | 45      |
| macro avg    | 1.00      | 1.00   | 1.00     | 45      |
| weighted avg | 1.00      | 1.00   | 1.00     | 45      |

## Result

The neural network is trained and tested successfully.

## Conclusion

Backpropagation enables the network to learn patterns and improve prediction accuracy.

## 7. Logistic Regression

### Aim

To implement Logistic Regression for classification.

### Algorithm

1. Load dataset.
2. Split training and testing data.
3. Train Logistic Regression model.
4. Predict outputs.
5. Calculate accuracy.

### **Python Code**

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
iris = load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3)
```

```
model = LogisticRegression(max_iter=200)
```

```
model.fit(X_train, y_train)
```

```
accuracy = model.score(X_test, y_test)
```

```
print("Accuracy:", accuracy)
```

### **Output**

```
LOGISTIC REGRESSION
=====
... Training Samples : 105
Testing Samples : 45

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Actual Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0 0 0 1 0 0 2 1
 0 0 0 2 1 1 0 0]

Accuracy = 100.0 %

Confusion Matrix
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]

Classification Report
              precision    recall  f1-score   support

     0           1.00       1.00       1.00         19
     1           1.00       1.00       1.00         13
     2           1.00       1.00       1.00         13

 accuracy          1.00
 macro avg         1.00
weighted avg         1.00
```

## Result

Classification accuracy is obtained.

## Conclusion

Logistic Regression effectively performs classification tasks.

## 8. Linear Regression

### Aim

To implement Linear Regression for predicting continuous values.

### Algorithm

1. Load dataset.

2. Split dataset.
3. Train Linear Regression model.
4. Predict outputs.
5. Evaluate model.

### **Python Code**

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np
```

```
X = np.array([[1],[2],[3],[4],[5]])
y = np.array([2,4,6,8,10])
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2)
```

```
model = LinearRegression()
```

```
model.fit(X_train, y_train)
```

```
prediction = model.predict(X_test)
```

```
print("Prediction:", prediction)
```

### **Output**

```
LINEAR REGRESSION
=====
... Training Data:
[ 6  1  8  3 10  5  4  7]

Testing Data:
[9 2]

Predicted Values:
[18.  4.]

Actual Values:
[18  4]

Slope (m): 2.0000000000000004
Intercept (c): -1.7763568394002505e-15

Mean Squared Error: 3.944304526105059e-31
R2 Score: 1.0
```

## Result

Predicted values are generated successfully.

## Conclusion

Linear Regression models the relationship between variables using a straight line.

---

## 9. Comparison of Linear and Polynomial Regression

### Aim

To compare Linear Regression and Polynomial Regression.

### Algorithm

1. Load dataset.
2. Train Linear Regression.
3. Transform data using Polynomial Features.
4. Train Polynomial Regression.
5. Compare results.

## Python Code

```
import numpy as np

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

X = np.array([[1],[2],[3],[4],[5]])
y = np.array([1,4,9,16,25])

linear = LinearRegression()
linear.fit(X,y)

poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X)

poly_model = LinearRegression()
poly_model.fit(X_poly,y)

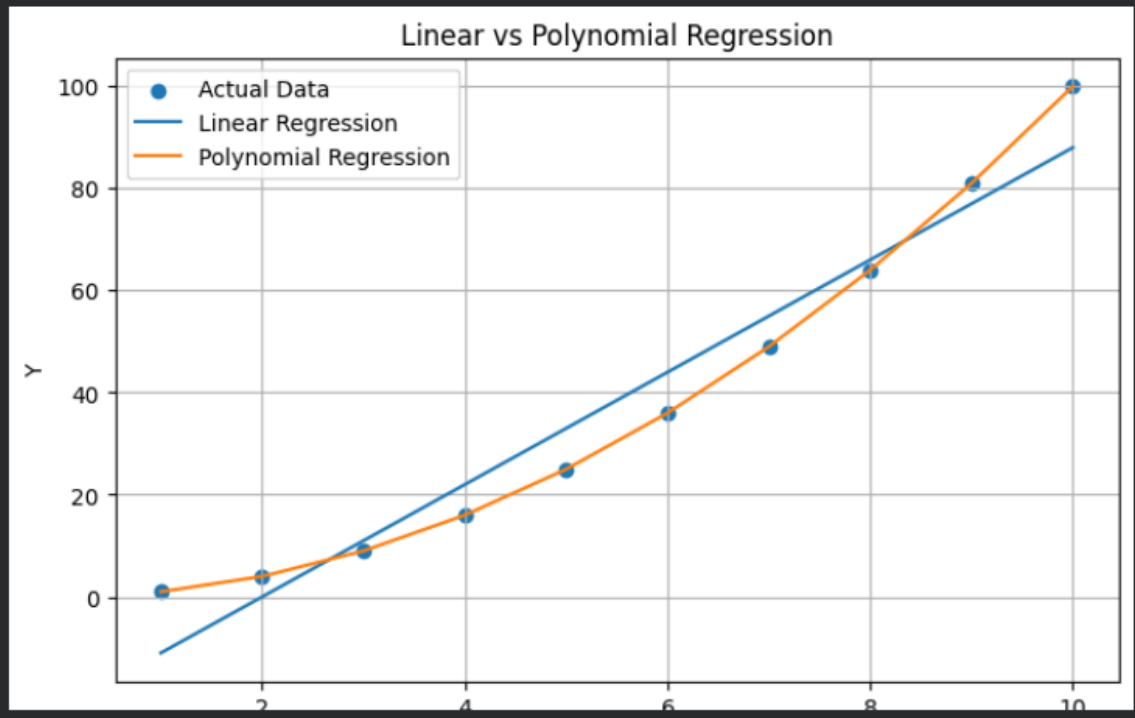
print("Linear Score:", linear.score(X,y))
print("Polynomial Score:",
      poly_model.score(X_poly,y))
```

## Output

### ... COMPARISON OF LINEAR AND POLYNOMIAL REGRESSION

Linear Regression  $R^2$  Score:  
0.9498

Polynomial Regression  $R^2$  Score:  
1.0



## Result

Polynomial Regression achieves a better fit for nonlinear data.

## Conclusion

Polynomial Regression is preferred when the relationship between variables is nonlinear.

---

## 10. Expectation Maximization (EM) Algorithm

### Aim

To implement the Expectation Maximization Algorithm for clustering.

### Algorithm

1. Initialize parameters.
2. Perform Expectation Step.

3. Perform Maximization Step.
4. Update parameters.
5. Repeat until convergence.

### **Python Code**

```
from sklearn.mixture import GaussianMixture
from sklearn.datasets import load_iris
```

```
iris = load_iris()
```

```
X = iris.data
```

```
gmm = GaussianMixture(
    n_components=3,
    random_state=42
)
```

```
gmm.fit(X)
```

```
labels = gmm.predict(X)
```

```
print("Cluster Labels:")
```

```
print(labels)
```

### **Output**



```

... EXPECTATION MAXIMIZATION ALGORITHM
=====

First 20 Cluster Labels:
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]

First 20 Actual Class Labels:
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]

Sample Results:
  Actual_Class Predicted_Cluster
0             0                 1
1             0                 1
2             0                 1
3             0                 1
4             0                 1
5             0                 1
6             0                 1
7             0                 1
8             0                 1
9             0                 1

Number of Clusters: 3

Means of Clusters:
[[6.54639415 2.94946365 5.48364578 1.98726565]
 [5.006      3.428      1.462      0.246      ]
 [5.9170732  2.77804839 4.20540364 1.29848217]]

Weights of Clusters:
[0.36539575 0.33333333 0.30127092]

```

## Result

Data points are grouped into clusters using the EM algorithm.

## Conclusion

The EM algorithm effectively performs probabilistic clustering by iteratively refining model parameters.

